FLUX: A Declarative Language for AI Ecosystems

A Minimal, Portable Substrate for Modular AI Pipelines

Luciano Taddei
Independent Researcher

May 17, 2026

Abstract

Modern production AI systems are complex ecosystems of heterogeneous models, retrieval modules, and human-in-the-loop checkpoints. Yet no standard, declarative language exists to describe and execute such pipelines in a portable, backend-agnostic way. We present **FLUX** (Flexible Language for Unified eXecution), a formal architectural description language for AI ecosystems. FLUX defines three core constructs—`ENTITY` (computational actors), `ATTRACTOR` (intent-routed pipelines), and `ECOSYSTEM` (versioned containers)—organized into a recursive, fractal hierarchy. The accompanying FLUX kernel (v2.2.0-alpha, 100KB Python) acts as a **universal substrate**: it parses ecosystem descriptions, enforces per-tenant budgets, provides automatic retry/fallback, adapts stage temperatures, and exports Prometheus metrics. A key design goal is portability: we demonstrate the kernel running stably on a resource-constrained machine (HP Z420, 20GB RAM, CPU-only, no AVX2), orchestrating models ranging from TinyLlama to Llama for production and from Hermes to DeepSeek Coder for architectural reasoning. While preliminary, this test validates FLUX as a lightweight, hardware-agnostic foundation for AI orchestration. We release the kernel as open-source software and propose FLUX as a candidate for a community-driven standard description language for modular AI pipelines.

Contents

1 Introduction

1.1 The Orchestration Gap

The shift from monolithic models to compound AI systems—pipelines that chain multiple LLMs, retrieval steps, and human-in-the-loop interventions—has created a pressing need for principled orchestration. Yet current practice relies almost exclusively on two approaches, neither of which is fully satisfactory:

1. **Framework-coupled scripting**: LangChain, Haystack, Semantic Kernel, and DSPy offer powerful Python APIs to define pipelines. However, these pipelines are not portable across frameworks, and the orchestration logic is entangled with platform-specific code, making it difficult to audit, version, or migrate.

2. **Ad-hoc Python scripts**: Many production systems bypass frameworks entirely, wiring together LLM API calls with custom logic. Such scripts are brittle, lack standardized observability, and provide no guarantees about budget enforcement or reproducibility.

What is missing is a **declarative, framework-agnostic language** for describing AI ecosystems—a language that cleanly separates *what* an AI system is from *how* it is executed, and that can serve as a **common substrate** across different models, backends, and hardware environments.

1.2 FLUX: A Universal Substrate

FLUX (Flexible Language for Unified eXecution) is designed to fill this gap. It consists of:

- A **declarative grammar** (the `.flux` format) that describes entities, intent-routed pipelines, and global policies.
- A **minimal execution kernel** that parses these descriptions and executes them on any supported backend, providing budget control, retry/fallback, observability, and adaptive temperature tuning.

FLUX is deliberately **minimal and stable**. It does not include automatic pipeline synthesis, prompt optimization, or complex memory management—these are left to future extensions or external tools. Instead, it provides a **universal substrate**: a lightweight, predictable layer that can load, execute, and observe any AI ecosystem described in the FLUX language. The kernel is designed to run even on hardware with extremely limited resources, as demonstrated in our preliminary validation (Section 5).

1.3 Contributions

This paper contributes:

1. The formal definition of the FLUX language, its grammar, and its layers of abstraction (Section 3).
2. The architecture of the FLUX kernel v2.2.0-alpha, a reference implementation released as open-source software (Section 4).
3. A preliminary experimental validation demonstrating stable execution on constrained hardware with heterogeneous models (Section 5).
4. A discussion of current limitations and a roadmap for community-driven standardization (Sections 6–7).

2 Related Work

FLUX intersects several active areas of research and development.

Pipeline Orchestration Frameworks. LangChain [1], Haystack [2], and Semantic Kernel [3] provide Python APIs for chaining LLM calls, retrieving documents, and integrating tools. They excel at rapid prototyping but couple pipeline definition to their specific execution model. FLUX, by contrast, provides a declarative, framework-agnostic description that can be executed by any runtime implementing the FLUX specification.

Declarative Prompting Languages. DSPy [4] introduces a programming model where LLM calls are abstracted as declarative modules with automatically optimized prompts. While sharing the declarative spirit, DSPy focuses on *prompt optimization*, whereas FLUX focuses on *pipeline architecture and execution*. The two approaches are complementary.

Graph-Based Reasoning. Tree-of-Thoughts [5], Graph-of-Thoughts [6], and Adaptive Graph of Thoughts [7] model reasoning as graph topologies. FLUX's `ATTRACTOR` construct generalizes these to arbitrary directed acyclic graphs with cross-stage observations (`OBSERVE`), while its recursive `ECOSYSTEM` composition enables hierarchical reasoning at multiple scales.

Multi-Agent Architectures. AutoGen [8] and CrewAI [9] orchestrate multiple AI agents with specialized roles. FLUX can describe such multi-agent systems declaratively, with each agent as an `ENTITY` and their interaction patterns as `ATTRACTOR`s.

Formal ADLs. Architecture Description Languages (ACME, AADL, SysML) have been used for decades to specify software and hardware systems. FLUX adapts the ADL paradigm to the specific challenges of AI ecosystems: managing model-specific parameters (temperature, quantization), tracking inference costs, and enabling runtime adaptation.

Serving Infrastructure. BentoML, MLflow, and Seldon Core focus on model serving and deployment. FLUX operates at a higher abstraction level, describing the *topology and policy* of an AI system rather than its deployment configuration.

Table 1 provides a structured comparison.

Table 1: Comparison of FLUX with existing approaches. • = full support, ○ = partial/plugin, × = absent.

Approach	Declar.	Backend-agn.	Multi-ten.	Self-adapt.	Fractal	Std. Grammar
LangChain/Haystack	○	×	○	×	×	×
DSPy	•	○	×	• (prompts)	×	×
Graph-of-Thoughts	×	×	×	×	×	×
AutoGen/CrewAI	×	×	×	×	×	×
BentoML/MLflow	×	•	○	×	×	×
FLUX	•	•	•	○ (temper.)	•	•

3 The FLUX Language

3.1 Design Principles

FLUX rests on three principles: (1) **separation of concerns** between ecosystem description and backend execution; (2) **recursive composability** of all constructs; and (3) **observability by default**, with metrics automatically collected and exposed.

3.2 Grammar

A FLUX document is a text file with the `.flux` extension. Its syntax is defined by a context-free grammar implemented with the Lark parser [10]. Figure 1 shows the core production rules.

```
ecosystem: ECOSYSTEM_KW ESCAPED_STRING "{"
    (entity_def | attractor_def | policy_def)* "}"

entity_def: ENTITY_KW ESCAPED_STRING "{" entity_body "}"
entity_body: (type_def | nature_def | quantum_state_def
             | costo_def | fallback_def | model_key_def)*

attractor_def: ATTRACTOR_KW ESCAPED_STRING "{"
    on_intent_def stage_def+ "}"
stage_def: STAGE_KW ESCAPED_STRING "{" stage_body "}"
stage_body: (execute_def | temperature_def
            | max_new_tokens_def | prompt_transform_def
            | observe_def | ground_truth_def)*
```

Listing 1: Core FLUX grammar (excerpt).

Figure 1: Core production rules of the FLUX grammar.

3.3 Core Constructs and Abstraction Levels

FLUX organizes its constructs into a nested hierarchy (Table 2).

Table 2: Abstraction levels in FLUX.

Level	Construct	Role
Macro	`ECOSYSTEM`	Top-level container, versioning, global policy
Meso	`ATTRACTOR`	Intent-routed pipeline, stage sequence
Micro	`STAGE`	Single model invocation, parameters
Atomic	`ENTITY`	Individual model, human, or sub-ecosystem

- `ENTITY`: The computational actor. Its `TYPE` field specifies whether it is a single model, an ensemble, a human, or recursively another `ECOSYSTEM`. Key attributes include `MODEL_KEY` (backend identifier), `COSTO`, `FALLBACK`, and optional `RECURSIVE_CONFIG` for long-context processing.
- `ATTRACTOR`: A pipeline triggered by one or more user intents (`ON_INTENT`). It contains an ordered list of `STAGE`s, each defining the entity to invoke (`EXECUTE`), its temperature, max tokens, and optional prompt transformation or cross-stage observation.
- `ECOSYSTEM`: The top-level container that groups all entities and attractors, and declares a global `POLICY` (budget, latency limits, judge entity).

This hierarchy is **fractal** because an `ENTITY` of type `ECOSYSTEM` is itself a complete FLUX ecosystem, enabling recursive composition of arbitrarily complex pipelines.

4 The FLUX Kernel

The FLUX kernel v2.2.0-alpha is a 100KB Python implementation that serves as the reference runtime. It is designed as a **minimal, stable substrate** rather than a full-featured orchestration platform.

4.1 Architecture

The kernel includes:

- **Parser** (`FluxParser`): a LALR(1) parser built with Lark that generates Python objects from `.flux` files.
- **Interpreter** (`EcosystemInterpreter`): executes attractors sequentially, with EMA-based adaptive temperature, exponential backoff for transient errors, a fallback chain (up to depth 5), and automatic recursive processing for prompts exceeding the model's context window.
- **Loader** (`FluxModelLoader`): a pluggable backend abstraction. Currently supports vLLM (OpenAI-compatible), HuggingFace TGI, Ollama (local models), and a stub backend for testing.
- **Policy Manager** (`FluxPolicy`): enforces per-tenant budget, latency, and human-iteration limits atomically before each inference.
- **Mutation Engine** (`MutationEngine`): adjusts stage temperatures when confidence drops below a threshold. This is intentionally simple; complex auto-mutation is deferred to future extensions.
- **Multi-Tenancy** (`TenantManager`): supports isolated tenants with independent ecosystems, budgets, and model mappings.
- **Observability** (`FluxMetrics`): exposes Prometheus metrics (counters, histograms, gauges) for requests, errors, latency, confidence, temperature, and cost.

4.2 Portability and Hardware Agnosticism

A key design goal of the FLUX kernel is the ability to run on **hardware with extremely limited resources**. The kernel itself has negligible CPU and memory requirements, and it delegates all model inference to the configured backend. This means that an ecosystem can orchestrate a mix of lightweight local models (e.g., for simple queries) and heavyweight cloud models (for complex reasoning), with the kernel providing a uniform execution environment across both. Section 5 demonstrates this portability in practice.

5 Preliminary Validation on Constrained Hardware

To assess the stability and portability of the FLUX kernel under realistic constraints, we conducted a validation run on an **HP Z420 workstation** with the following specifications:

- CPU: Intel Xeon E5-1620 (4 cores, 3.60GHz, **no AVX2**).
- RAM: 20 GB DDR3.
- GPU: None (CPU-only execution).
- Storage: Standard HDD.

This machine represents a deliberately limited environment, typical of edge deployments, legacy infrastructure, or low-budget development setups. All model inference was performed via the Ollama backend, using CPU-only execution.

5.1 Models and Pipeline

We configured a simple FLUX ecosystem with two entities and one attractor:

- **Production entity**: Switched between `tinyllama` (1.1B parameters, quantized 4-bit) for fast, low-cost responses, and `llama2-7b` (7B parameters, 4-bit) for higher-quality output.
- **Architect entity**: Switched between `hermes-2-pro-mistral` and `deepseek-coder-1.3b`, used for reasoning about ecosystem configurations (the future Meta-Architect role).
- A single `ATTRACTOR` routed all queries to the production entity, with the architect entity optionally invoked for plan generation.

The pipeline was executed continuously for approximately 50 runs under a simulated multi-tenant load, with per-tenant budget and latency limits enforced. Prometheus metrics were collected throughout.

5.2 Results

The kernel operated **without crashes or memory leaks** for the entire duration. Key observations:

- **Budget enforcement**: No tenant exceeded its allocated budget, and requests were correctly rejected when limits were reached.
- **Fallback chains**: When the primary model timed out (simulated), the system automatically fell back to the secondary entity as specified.
- **Adaptive temperature**: The EMA controller smoothly adjusted stage temperatures in response to confidence fluctuations.
- **Resource usage**: The kernel's own memory footprint remained below 200MB. Model inference was handled entirely by Ollama, with the kernel orchestrating the pipeline without additional overhead.
- **Backend switching**: Switching between `llama2-7b` and `tinyllama` required only a change in the `MODEL_KEY` field of the entity definition; the kernel adapted seamlessly.

These results, while preliminary and not a formal benchmark, demonstrate that FLUX fulfills its design goal: it provides a stable, lightweight, and truly portable execution environment for AI ecosystems, even on hardware that lacks GPU acceleration or modern CPU features.

6 Discussion and Limitations

What FLUX is not. FLUX is not a model, a training framework, a prompt optimizer, or a model server. It is an architectural description language and a minimal runtime that executes those descriptions. It does not replace LangChain, DSPy, or BentoML; rather, it can serve as a **common target** into which pipelines defined in those frameworks can be compiled or translated.

Security. Because FLUX documents are executable code, they must be treated with the same caution as any script. The current kernel does not implement sandboxing. In production, FLUX files should be subject to code review, integrity verification, and access control.

Limitations. The current kernel has several limitations:

- **No Meta-Designer**: the system cannot automatically generate or mutate ecosystem configurations except for simple temperature adjustments.

- **No Calibrated Judge**: the built-in judge uses a generic LLM prompt, which may introduce self-preference bias.
- **Linear Stage Execution**: stages are executed sequentially; true fan-out/fan-in parallelism is not yet implemented.
- **Single-Process Runtime**: the kernel runs in a single OS process and does not support distributed execution.

Toward a Standard. FLUX is released under the MIT license and will be made available at a public repository upon publication. We invite the community to contribute alternative implementations, suggest language extensions, and participate in a possible RFC process to evolve the grammar into a community standard.

7 Conclusion and Future Work

We have presented FLUX, a declarative language and minimal execution kernel for AI ecosystems. FLUX addresses the need for a **portable, framework-agnostic description language** that can serve as a common substrate for modular AI pipelines. Its formal grammar, recursive composability, and pluggable backend architecture make it adaptable to virtually any existing AI system, while its built-in observability, multi-tenancy, and budget enforcement provide production-ready features.

Our preliminary validation on severely constrained hardware demonstrates that FLUX is not merely a conceptual proposal: it is a working, stable, and lightweight runtime that can orchestrate heterogeneous models even without GPU acceleration.

Future work will focus on:

1. Developing a **Meta-Designer** module for automated ecosystem synthesis.
2. Implementing **parallel stage execution** with fan-out/fan-in patterns.
3. Calibrating the judge entity on human-annotated quality assessments.
4. Extending the kernel to support **distributed execution**.
5. Engaging the community to evolve FLUX into a formal standard.

References

[1] LangChain. (2023). *Building applications with LLMs through composability.* `https://github.com/langchain-ai/langchain`

[2] Haystack. (2023). *NLP framework to build applications with LLMs.* `https://github.com/deepset-ai/haystack`

[3] Microsoft. (2023). *Semantic Kernel.* `https://github.com/microsoft/semantic-kernel`

[4] Khattab, O. et al. (2023). *DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines.* arXiv:2310.03714.

[5] Yao, S. et al. (2023). *Tree of Thoughts.* arXiv:2305.10601.

[6] Besta, M. et al. (2024). *Graph of Thoughts.* arXiv:2308.09687.

[7] Pandey, T. et al. (2025). *Adaptive Graph of Thoughts.* arXiv:2502.05078.

[8] Wu, Q. et al. (2023). *AutoGen*. arXiv:2308.08155.

[9] CrewAI. (2024). *Framework for orchestrating role-playing AI agents*. `https://github.com/crewAIInc/crewAI`

[10] Lark. *A modern parsing library for Python*. `https://github.com/lark-parser/lark`

[11] Shumailov, I. et al. (2024). *AI models collapse when trained on recursively generated data*. Nature, 631(8022), 755-759.

[12] He, H., Xu, S., & Cheng, G. (2024). *Golden Ratio Weighting Prevents Model Collapse*. arXiv preprint.

[13] Jaeger, S. (2021). *A Dual Process Model for Optimizing Cross Entropy in Neural Networks*. arXiv:2104.13277.